Netskope Client Troubleshooting & Diagnostics Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Netskope Client Troubleshooting & Diagnostics Toolkit 4
- What you get . . . 4
- What this delivers . . . 4
- Vendor boundary . . . 4
- The deployment model . . . 4
- Outcome . . . 4

1. Solution Overview — Netskope Client Troubleshooting & Diagnostics Toolkit 5
- What this solution is . . . 5
- The why (this section only) . . . 5
- The Netskope client from a support view . . . 5
- The four failure classes . . . 5
- The diagnose-then-resolve model . . . 6
- Naming convention (used everywhere) . . . 6
- Benefits . . . 6
- Limitations (be honest with buyers) . . . 6
- The deployment model . . . 7

2. Requirements Guide — Netskope Client Troubleshooting & Diagnostics Toolkit 7
- 2.1 Netskope client deployed on endpoints . . . 7
- 2.2 Local administrator rights on the endpoint . . . 7
- 2.3 Access to client logs (for handoff) . . . 7
- 2.4 Microsoft Graph access for device context . . . 7
- 2.5 Network reachability from the endpoint . . . 7
- 2.6 A diagnostics scope group (help-desk pilot) . . . 8
- 2.7 Change control and a rollback path . . . 8
- Checklist . . . 8

3. Implementation Guide — Netskope Client Troubleshooting & Diagnostics Toolkit 8
- Step 1 — Assess: confirm prerequisites and baseline . . . 8
- Step 2 — Stand up the diagnostics toolkit . . . 9
- Step 3 — Build the per-failure-class runbooks . . . 9
- Step 4 — Wire log/state collection . . . 10
- Step 5 — Configure the pilot scope . . . 10
- Step 6 — Pilot with the help desk . . . 10
- Step 7 — Validate the pilot (gate) . . . 10
- Step 8 — Expand to the wider support org . . . 10

5. Testing Guide — Netskope Client Troubleshooting & Diagnostics Toolkit 11

Test 1 — Discovery collects client state . . . 11
Test 2 — Validation checks connectivity . . . 11
Test 3 — Report returns device context . . . 11
Test 4 — Remediation is safe by default . . . 11
Test 5 — Scope group builds . . . 11
Test 6 — Steering runbook resolves its class . . . 11
Test 7 — Authentication runbook classifies correctly . . . 11
Test 8 — Clean escalation . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Netskope Client Troubleshooting & Diagnostics Toolkit 12
Part A — Failure-class runbooks (diagnose then resolve) . . . 12
Part B — Troubleshooting the solution itself . . . 13
Part C — When to escalate to Netskope support . . . 14
7. Best Practices — Netskope Client Troubleshooting & Diagnostics Toolkit 14
8. Deliverables Summary — Netskope Client Troubleshooting & Diagnostics Toolkit 15
Documentation . . . 15
PowerShell scripts . . . 15
Named artifacts (reused verbatim) . . . 16
What the buyer receives . . . 16
Vendor boundary . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Netskope Client Troubleshooting & Diagnostics Toolkit

When the Netskope client misbehaves — traffic not steering, an authentication loop, slow browsing, or an application broken behind the proxy — the help desk usually burns hours guessing. Tickets bounce between desktop, network, and security teams because nobody has a repeatable way to collect the right state and read it. This solution replaces that guesswork with a **diagnose-then-resolve** toolkit: structured runbooks for the common failure classes plus read-only diagnostics that gather the client state, services, connectivity, and device context that point to the root cause.

This is a complete, deploy-it-today package for troubleshooting the Netskope client, written for a help-desk technician or endpoint engineer with no prior Netskope-internals experience. It is the repeatable way to run client triage at scale.

What you get

- A plain-language **Solution Overview** of the client from a support view, the four failure classes (steering, authentication, performance, certificate/decryption), and the diagnose-then-resolve model.
- A **Requirements Guide** with Verify / Implement-if-missing for every prerequisite.
- An **Implementation Guide** that stands up the toolkit, builds the per-failure-class runbooks, and rolls them out: Assess → Configure → Pilot (help desk) → Validate → Expand.
- Five **PowerShell scripts** for client-state discovery, steering/auth/connectivity validation, a client-health report, safe remediation (restart/repair/re-enroll the client, default `-WhatIf`), and a diagnostics-scope group builder.
- **Testing** and **Troubleshooting** guides with explicit pass/fail gates.

What this delivers

A consistent triage workflow: every technician collects the same diagnostic bundle, follows the same named runbook for the symptom in front of them, and either resolves at tier 1 or escalates with a complete evidence package — so issues are fixed at root cause instead of by reboot-and-hope.

Vendor boundary

Netskope console screens and client-internal log paths change over time and are **deferred to the official Netskope documentation and your tenant**. This toolkit operates at the failure-class and operating-system level — services, processes, network reachability, certificates, and device context via Microsoft Graph — and flags where to hand off to vendor docs for the precise control.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand**, gated by a help-desk pilot.

Outcome

Netskope client issues resolved fast — failure-class runbooks and diagnostics that collect the right state and point to root cause — so the help desk stops guessing and tickets close at first contact.

1. Solution Overview — Netskope Client Troubleshooting & Diagnostics Toolkit

What this solution is

This is a troubleshooting and diagnostics toolkit for the Netskope client on managed endpoints. It is not a Netskope deployment guide — it assumes the client is already deployed and gives the help desk a repeatable way to triage when it breaks: structured runbooks for the common failure classes, plus read-only diagnostics that collect the right operating-system state and device context so a technician can find the root cause instead of guessing.

The why (this section only)

Most Netskope client tickets fail not because the problem is hard but because triage is inconsistent. One technician reboots, another reinstalls, a third escalates immediately — and the evidence that would identify the fault is never collected. The cost is long ticket lifetimes, needless re-enrollments, and security teams pulled into desktop problems. A diagnose-then-resolve model fixes this: collect a standard diagnostic bundle, match the symptom to a named runbook, attempt the safe remediation for that class, and escalate only with a complete evidence package. Consistency is the value — the same inputs, the same decisions, every ticket.

The Netskope client from a support view

The client is a managed endpoint agent that **steers** selected traffic to the Netskope cloud (inline secure web gateway and/or Zero Trust network access), **authenticates** the user/device so identity-based policy can apply, and participates in **SSL/TLS inspection** by presenting a trusted decryption certificate. From the operating system's perspective this surfaces as one or more **services and processes**, **network connectivity** to the cloud, **certificates** in the trust store, and **configuration/enrollment state**. Those four observable surfaces are exactly what the diagnostics in this toolkit read. The client's own internal logs and console-side configuration are **deferred to Netskope's documentation** — this toolkit describes the failure-class approach and points to where vendor detail takes over.

The four failure classes

Every common client ticket falls into one of four classes. Each has a dedicated runbook (see the Implementation and Troubleshooting guides).

1. **Steering** — traffic that should go through Netskope isn't (or traffic that shouldn't is). The client is not running, the configuration didn't apply, or an exception/bypass is wrong.
1. **Authentication** — the user/device can't establish or keep a session: enrollment failed, an identity/SSO loop, an expired token, or time/clock skew.
1. **Performance** — steering works but browsing or an app is slow: path/latency to the cloud,

inspection overhead on large transfers, or DNS resolution delay.

1. **Certificate / decryption** — TLS errors, untrusted-certificate warnings, or a pinned app that refuses inspection: the decryption root isn't trusted on the device, or the app needs a bypass.

The diagnose-then-resolve model

Every runbook follows the same shape so the workflow is identical regardless of class:

1. **Collect** — run the read-only diagnostics (Discovery, then Validation, then Report) to capture client state, services/processes, connectivity, certificates, and device context.
1. **Classify** — match the collected evidence to one of the four failure classes.
2. **Resolve** — apply the safe remediation for that class (sync/restart/repair/re-enroll, all `-WhatIf` by default), or apply the documented configuration fix.
1. **Escalate with evidence** — if unresolved at tier 1, hand off the diagnostic bundle (and the Netskope-specific client logs, per vendor docs) so the next tier starts from facts.

Naming convention (used everywhere)

Every artifact in this toolkit is named `<Purpose> – <Scope> – <Stage>` and the names are reused verbatim across the documents and scripts. Examples: `Client Triage – Steering – Diagnose`, `Client Triage – Authentication – Resolve`, `Diagnostic Bundle – Endpoint – Collect`, `Diagnostics Scope – Help Desk – Pilot`. The convention makes a runbook, a bundle, and a group unambiguous in a ticket.

Benefits

- **First-contact resolution** — a technician follows a named runbook instead of improvising.
- **Consistent evidence** — every ticket carries the same diagnostic bundle.
- **Less needless churn** — re-enrollment becomes a last resort, not a reflex.
- **Clean escalations** — security/network teams receive facts, not symptoms.
- **Repeatable at scale** — assign-once scope, run-everywhere scripts.

Limitations (be honest with buyers)

- This toolkit triages the **client and the endpoint's observable state**; it does not configure or reach into the Netskope cloud console.
- **Netskope-specific log paths and console menus are deferred** to vendor docs — they change over versions and tenants and are not fabricated here.
- Microsoft Graph reporting requires the Graph SDK and read scopes (noted in Requirements).
- Deep cloud-side faults (a tenant policy bug, a Netskope point-of-presence outage) are diagnosed to

the boundary and then escalated to Netskope support.

The deployment model

> ***Stand up the toolkit → build the per-class runbooks → pilot with the help desk → validate → expand to the wider support org.*** *Pattern:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Netskope Client Troubleshooting & Diagnostics Toolkit

Verify **every** requirement before running the toolkit at scale. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-network.md`.

2.1 Netskope client deployed on endpoints

Description: The Netskope client is installed and intended to run on the target endpoints. **Why:** This toolkit triages an existing client; there is nothing to diagnose without it. **Verify:** Run `NSDIAG-Discovery.ps1`; it reports the client's installed state, services, and processes. **Implement:** Deploy the client through Intune (or your endpoint manager) per Netskope's deployment docs, then re-run discovery.

2.2 Local administrator rights on the endpoint

Description: The technician (or a script-run context) has local admin on the device under triage. **Why:** Reading service state and certificates, and any restart/repair/re-enroll remediation, requires elevation. **Verify:** Run an elevated PowerShell session; `NSDIAG-Discovery.ps1` completes without access-denied warnings. **Implement:** Grant local admin for the support context (e.g. a privileged help-desk account or a managed elevation tool).

2.3 Access to client logs (for handoff)

Description: The ability to locate and collect the Netskope client's own log files for escalation. **Why:** Cloud/console-side faults are diagnosed to the boundary, then escalated with vendor-specific logs. **Verify:** Confirm the client log location per Netskope's documentation for your client version. **Implement:** Document the log path from Netskope docs in your runbook; `NSDIAG-Report.ps1` flags this as a vendor-deferred collection step. (Do not hard-code a fabricated path.)

2.4 Microsoft Graph access for device context

Description: Read access to Microsoft Graph for managed-device context (compliance, last sync, OS). **Why:** Device context distinguishes a client fault from an enrollment/compliance fault and helps fleet triage. **Verify:** `NSDIAG-Report.ps1` can `Connect-MgGraph` with `DeviceManagementManagedDevices.Read.All`. **Implement:** Install the Microsoft Graph PowerShell SDK and grant the read scope to the operator/app.

2.5 Network reachability from the endpoint

Description: The endpoint can resolve DNS and reach the internet / the Netskope cloud edge. **Why:** Steering, authentication, and inspection all depend on reachable, resolvable, routable traffic. **Verify:** `NSDIAG-Validation.ps1` runs DNS and TCP reachability checks and reports results. **Implement:** Remediate DNS/egress; confirm the Netskope edge hostnames/ports (per Netskope docs) are allowed through any upstream proxy/firewall.

2.6 A diagnostics scope group (help-desk pilot)

Description: A Microsoft Entra security group scoping the pilot (help-desk devices/users). **Why:** The toolkit is piloted with the help desk before wider rollout (deployment model gate). **Verify:** Entra → **Groups**; the `Diagnostics Scope – Help Desk – Pilot` group exists, or run `NSDIAG-Set-ScopeTargeting.ps1` to create/confirm it. **Implement:** Create the group via Entra or `NSDIAG-Set-ScopeTargeting.ps1` and add the pilot members.

2.7 Change control and a rollback path

Description: A change window and documented rollback for any remediation that restarts/repairs/re-enrolls the client. **Why:** Restart/repair/re-enroll touch live steering and must be reversible and communicated. **Verify:** A rollback (re-enroll undo / client reinstall / steering exception) is written and approved before remediation. **Implement:** Define the rollback and run remediation with the default `-WhatIf` first to preview impact.

Checklist

- Netskope client deployed — `NSDIAG-Discovery.ps1`
- Local admin on the endpoint
- Client log location documented for handoff (from Netskope docs)
- Microsoft Graph SDK + read scope for device context
- DNS/egress and Netskope edge reachability — `NSDIAG-Validation.ps1`
- `Diagnostics Scope – Help Desk – Pilot` group created
- Change control + rollback approved for remediation

3. Implementation Guide — Netskope Client Troubleshooting & Diagnostics Toolkit

Follow these steps **in order**. The goal is a standing triage capability — diagnostics deployed, the four runbooks written and named, piloted with the help desk, validated, then expanded.

> ***Model:*** *Stand up the diagnostics, build the per-failure-class runbooks, wire collection, pilot with the help desk, validate against known issues, then expand. Pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

Step 1 — Assess: confirm prerequisites and baseline

1. Complete the Requirements checklist (Section 2) and the category global requirements.

2. On a representative endpoint, run `NSDIAG-Discovery.ps1` to capture the baseline client state (installed/services/processes), then `NSDIAG-Validation.ps1` for connectivity. Keep the output as the `Diagnostic Bundle – Endpoint – Collect` baseline.

Step 2 — Stand up the diagnostics toolkit

1. Place the five scripts in a controlled location available to the help desk (a managed share, an Intune script package, or a remediation tool).

1. Confirm execution policy and elevation: `powershell -ExecutionPolicy Bypass -File .\NSDIAG-Discovery.ps1`.

2. Confirm Microsoft Graph reporting works once: `NSDIAG-Report.ps1` connects with `DeviceManagementManagedDevices.Read.All`.

Step 3 — Build the per-failure-class runbooks

Author one runbook per class, each named with the convention and each following diagnose-then-resolve. Reuse these names verbatim:

- `Client Triage – Steering – Diagnose` / `Client Triage – Steering – Resolve`
- `Client Triage – Authentication – Diagnose` / `Client Triage – Authentication – Resolve`
- `Client Triage – Performance – Diagnose` / `Client Triage – Performance – Resolve`
- `Client Triage – Certificate – Diagnose` / `Client Triage – Certificate – Resolve`

Each runbook's structure:

1. **Collect** the `Diagnostic Bundle – Endpoint – Collect` (`NSDIAG-Discovery.ps1` → `NSDIAG-Validation.ps1` → `NSDIAG-Report.ps1`).

1. **Classify** using the decision cues below.

2. **Resolve** with the safe remediation, then re-validate.

3. **Escalate** with the bundle plus the Netskope client logs (vendor-deferred path) if unresolved.

Classification cues (what the bundle shows per class):

- **Steering:** client service/process not running, or running but no steering config applied / a bypass present. Resolve: `NSDIAG-Remediation.ps1` restart/repair; if still wrong, re-enroll.

- **Authentication:** enrollment incomplete, identity loop, clock skew. Resolve: correct time, then `NSDIAG-Remediation.ps1` re-enroll; confirm device context in `NSDIAG-Report.ps1`.

- **Performance:** steering healthy but high latency / DNS delay on the validation checks. Resolve:

per `Client Triage – Performance – Resolve` (path/inspection bypass per Netskope docs).

• **Certificate:** decryption root not trusted or TLS errors in validation. Resolve: confirm the decryption certificate is present/trusted (deferred to Netskope deployment docs for the cert source).

Step 4 — Wire log/state collection

1. Make `NSDIAG-Discovery.ps1`, `NSDIAG-Validation.ps1`, and `NSDIAG-Report.ps1` runnable in one pass (a wrapper or a documented sequence) to produce the `Diagnostic Bundle – Endpoint – Collect`.

1. Document, from Netskope's docs, where the client's own logs live for your client version, so the bundle handoff is complete. `NSDIAG-Report.ps1` prints a reminder for this vendor-deferred step.

Step 5 — Configure the pilot scope

1. Create the `Diagnostics Scope – Help Desk – Pilot` group (Entra, or `NSDIAG-Set-ScopeTargeting.ps1`).
2. Add the help-desk technicians' devices/accounts as pilot members.

Step 6 — Pilot with the help desk

1. Have the help desk run live tickets through the four runbooks for one cycle.
2. For each ticket: collect the bundle, classify, attempt the safe remediation with `-WhatIf` first, then apply. Record outcomes.

Step 7 — Validate the pilot (gate)

1. Run the Testing Guide (Section 5) — confirm discovery, validation, report, and a `-WhatIf` remediation behave correctly, and that each runbook resolves or cleanly escalates its class.

1. **Gate:** do not expand until the help-desk pilot passes the Testing Guide gates.

Step 8 — Expand to the wider support org

1. Publish the toolkit and the four named runbooks to the broader support team.
2. Broaden the scope group from `Diagnostics Scope – Help Desk – Pilot` to the full support scope.
3. Re-validate each cycle with `NSDIAG-Report.ps1` (fleet device context) and keep the runbooks in sync with any Netskope client/console changes (re-confirm vendor-deferred paths after upgrades).

Order of operations

`Assess/baseline → stand up scripts → build 4 runbooks → wire collection → pilot scope → help-desk pilot → validate (gate) → expand`

5. Testing Guide — Netskope Client Troubleshooting & Diagnostics Toolkit

Run during the help-desk pilot (Implementation Step 7) and after any Netskope client/console change. These tests validate the toolkit itself — that diagnostics collect correctly and each runbook resolves or cleanly escalates its failure class.

Test 1 — Discovery collects client state

Do: On a target endpoint run `NSDIAG-Discovery.ps1`. **Expected:** Reports client installed state, service status, processes, and certificate presence; no access-denied errors (elevated).

Test 2 — Validation checks connectivity

Do: Run `NSDIAG-Validation.ps1`. **Expected:** DNS resolves, TCP reachability checks return a clear pass/fail, and a steering/auth state summary prints.

Test 3 — Report returns device context

Do: Run `NSDIAG-Report.ps1` and connect to Microsoft Graph. **Expected:** Per-device context (OS, last sync, compliance) prints; the vendor-deferred client-log reminder is shown.

Test 4 — Remediation is safe by default

Do: Run `NSDIAG-Remediation.ps1` with no switches. **Expected:** It runs in `-WhatIf` mode and only *describes* the restart/repair/re-enroll actions — it changes nothing until explicitly confirmed.

Test 5 — Scope group builds

Do: Run `NSDIAG-Set-ScopeTargeting.ps1` (with `-WhatIf` first). **Expected:** It previews, then on confirmation creates/confirms `Diagnostics Scope – Help Desk – Pilot` and reports membership.

Test 6 — Steering runbook resolves its class

Do: On a device with the client service stopped, run `Client Triage – Steering – Diagnose`, then the resolve path. **Expected:** Discovery shows the service down; the safe restart remediation brings steering back, confirmed by re-running Validation.

Test 7 — Authentication runbook classifies correctly

Do: On a device with an enrollment/auth issue, run `Client Triage – Authentication – Diagnose`. **Expected:** The bundle and device context point to enrollment/identity/clock skew; the runbook selects re-enroll (not a blind reinstall).

Test 8 — Clean escalation

Do: Simulate an unresolved case (e.g. a suspected cloud-side fault). **Expected:** The runbook produces the complete `Diagnostic Bundle – Endpoint – Collect` plus the vendor-log handoff note, ready for Netskope support.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 pass, and Tests 6–8 show at least one class resolved and one clean escalation
Expand pass	All eight tests pass and the help desk completed one live ticket cycle through the runbooks

6. Troubleshooting Guide — Netskope Client Troubleshooting & Diagnostics Toolkit

This product *is* a troubleshooting toolkit, so this section serves two purposes: (A) the failure-class runbooks themselves, and (B) troubleshooting the solution — when the toolkit or its scripts misbehave. Use `NSDIAG-Discovery.ps1` / `NSDIAG-Validation.ps1` for device state, `NSDIAG-Report.ps1` for device context, and `NSDIAG-Remediation.ps1` for safe fixes.

> ***Vendor boundary:*** *client-internal log paths and Netskope console menus are deferred to the official Netskope documentation. The runbooks below describe the failure-class approach and stop at the operating-system edge, then hand off to vendor docs/support with the diagnostic bundle.*

Part A — Failure-class runbooks (diagnose then resolve)

A1. `Client Triage – Steering – Diagnose/Resolve`

Symptom	Likely cause	Diagnose	Resolve
Traffic not steered to Netskope	Client service/process not running	`NSDIAG-Discovery.ps1` shows service stopped	`NSDIAG-Remediation.ps1 -Restart`
Running but no steering	Config not applied / stale	Discovery shows running but Validation shows no steering	`NSDIAG-Remediation.ps1 -Repair`; if unchanged, `-ReEnroll`
Some traffic bypasses	Exception/bypass set	Validation flags bypassed destinations	Review the bypass in the Netskope console (deferred to vendor docs)

A2. `Client Triage – Authentication – Diagnose/Resolve`

Symptom	Likely cause	Diagnose	Resolve
Auth/sign-in loop	Identity/SSO loop or stale session	`NSDIAG-Validation.ps1` reports auth state not established	Correct any clock skew, then `NSDIAG-Remediation.ps1 -ReEnroll`
Enrollment incomplete	Client never finished enrolling	Discovery shows partial enrollment state	`NSDIAG-Remediation.ps1 -ReEnroll`; confirm device context in `NSDIAG-Report.ps1`
Works on some devices only	Device not compliant/managed	`NSDIAG-Report.ps1` shows non-compliant/never-synced	Remediate compliance/enrollment in Intune; re-validate

A3. `Client Triage – Performance – Diagnose/Resolve`

Symptom	Likely cause	Diagnose	Resolve
Slow browsing with client on	Path/latency to the cloud edge	`NSDIAG-Validation.ps1` reports high TCP latency	Investigate the egress path; confirm nearest edge per Netskope docs
Slow first-byte / name lookups	DNS resolution delay	Validation reports slow DNS	Fix DNS (resolver/egress); re-test
Large transfers slow	Inspection overhead on a specific app	Steering healthy, app-specific slowness	Apply the documented inspection bypass for that app (deferred to Netskope console docs)

A4. `Client Triage – Certificate – Diagnose/Resolve`

Symptom	Likely cause	Diagnose	Resolve
Untrusted-certificate warnings	Decryption root not trusted on device	`NSDIAG-Discovery.ps1` shows the decryption cert missing from trust store	Deploy/trust the decryption root via Intune (cert source per Netskope deployment docs)
A pinned app breaks behind proxy	App refuses inspection (cert pinning)	TLS errors only for that app	Add the documented inspection bypass for the pinned app (deferred to Netskope docs)
TLS errors after a change	Cert rotated/expired	Validation reports TLS handshake failures	Confirm the current decryption cert is trusted; redeploy if rotated

Part B — Troubleshooting the solution itself

B1. Scripts won't run or report access denied

- Run elevated: `powershell -ExecutionPolicy Bypass -File .\NSDIAG-Discovery.ps1`.
- Confirm local admin (Requirement 2.2); read-only scripts still need elevation to read services/certs.

B2. `NSDIAG-Report.ps1` *can't connect to Graph*

- Install the Microsoft Graph SDK and grant `DeviceManagementManagedDevices.Read.All`.
- If consent fails, have an admin grant the scope, then retry `Connect-MgGraph`.

B3. Remediation seems to do nothing

- By design: `NSDIAG-Remediation.ps1` defaults to `-WhatIf` and only previews. Re-run with the explicit action switch (and a change window) to apply.

B4. Discovery can't find the client

- The client may not be installed (Requirement 2.1) — deploy it per Netskope docs and re-run, or the process/service names differ on your client version (confirm against Netskope docs and adjust the runbook note).

Part C — When to escalate to Netskope support

If the bundle shows the client running, steering applied, the device compliant, the decryption cert trusted, connectivity healthy, and the issue persists, the fault is likely cloud/console-side. Escalate with the complete `Diagnostic Bundle – Endpoint – Collect` **plus** the client's own logs (locate per Netskope's documentation for your client version) so Netskope support starts from facts.

7. Best Practices — Netskope Client Troubleshooting & Diagnostics Toolkit

- **Diagnose before you touch.** Always collect the `Diagnostic Bundle – Endpoint – Collect` (`NSDIAG-Discovery.ps1` → `NSDIAG-Validation.ps1` → `NSDIAG-Report.ps1`) before any remediation — the single most important habit. Evidence first, action second.
- **Classify, then follow the named runbook.** Match the symptom to one of the four classes (steering, authentication, performance, certificate) and run that runbook — don't improvise.
- **Make re-enroll a last resort.** Restart and repair fix most steering faults; reserve re-enroll for confirmed enrollment/auth faults. Reflexive reinstalls hide root causes and churn users.
- **Always preview with `-WhatIf`.** Run `NSDIAG-Remediation.ps1` and `NSDIAG-Set-ScopeTargeting.ps1` in `-WhatIf` first so the action is understood before it touches live steering.
- **Reuse the naming convention verbatim.** `<Purpose> – <Scope> – <Stage>` on every runbook, bundle, and group makes a ticket unambiguous and an escalation self-describing.

• **Respect the vendor boundary.** Diagnose to the edge of the operating system; for client-internal logs and console policy, hand off to Netskope's documentation rather than guessing paths/menus.

• **Escalate with a full bundle.** Never escalate symptoms — attach the diagnostic bundle plus the vendor-deferred client logs so the next tier starts from facts.

• **Check the clock on auth failures.** Time/clock skew is a common, cheap-to-fix cause of authentication loops — verify it before re-enrolling.

• **Re-baseline after upgrades.** After a Netskope client or console change, re-confirm vendor paths and re-run the Testing Guide so the runbooks stay accurate.

• **Pilot with the help desk first.** Validate the toolkit on real tickets with the help desk before expanding to the wider support org — the pilot gate keeps bad runbooks off the floor.

8. Deliverables Summary — Netskope Client Troubleshooting & Diagnostics Toolkit

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The client from a support view, the four failure classes, diagnose-then-resolve, naming convention
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Stand up diagnostics, build the four runbooks, pilot, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Failure-class runbooks and troubleshooting the solution itself
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSDIAG-Discovery.ps1`	Read-only. Reports client install state, services, processes, and decryption-cert presence
`NSDIAG-Validation.ps1`	Read-only. Checks DNS, TCP reachability, and steering/auth state; clear pass/fail

Script	What it does
`NSDIAG-Report.ps1`	Read-only. Device context via Microsoft Graph; flags the vendor-deferred client-log handoff
`NSDIAG-Remediation.ps1`	Safe remediation (default `-WhatIf`): restart / repair / re-enroll the client
`NSDIAG-Set-ScopeTargeting.ps1`	Creates/confirms the `Diagnostics Scope – Help Desk – Pilot` group (default `-WhatIf`)

Named artifacts (reused verbatim)

- Runbooks: `Client Triage – Steering – Diagnose/Resolve`, … `– Authentication –` …, … `– Performance –` …, … `– Certificate –` …
- Bundle: `Diagnostic Bundle – Endpoint – Collect`
- Group: `Diagnostics Scope – Help Desk – Pilot`

What the buyer receives

- This guide set as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Vendor boundary

Netskope console screens and client-internal log paths are deferred to the official Netskope documentation and your tenant; this toolkit operates at the failure-class and operating-system level.

Outcome

Netskope client issues triaged the same way every time — diagnostics that collect the right state and runbooks that point to root cause — so the help desk resolves at first contact or escalates with facts.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSDIAG-Discovery.ps1

Purpose: Read-only. Collects the endpoint's observable Netskope client state: installed product, related services and processes, and whether a decryption (SSL inspection) certificate is present in the trust store. This is the first input to the Diagnostic Bundle - Endpoint - Collect. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Discovery.ps1 EXPECTED RESULTS Installed-client summary, service status, running processes, and decryption-cert presence. Netskope-internal log paths are deferred to Netskope's documentation (flagged below). .NOTES Network category — client troubleshooting toolkit. Read-only. Uses only standard OS facilities. #> [CmdletBinding()] param()

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Discovery.ps1 EXPECTED RESULTS Installed-client summary, service status, running processes, and decryption-cert presence. Netskope-internal log paths are deferred to Netskope's documentation (flagged below). .NOTES Network category — client troubleshooting toolkit. Read-only. Uses only standard OS facilities. #> [CmdletBinding()] param()

NSDIAG-Remediation.ps1

Purpose: Applies the safe remediation for a classified failure. SUPPORTS -WhatIf AND DEFAULTS TO IT: with no action confirmed it only PREVIEWS what it would do and changes nothing. Choose one action switch (-Restart / -Repair / -ReEnroll). Restart/repair are the first line; re-enroll is the last resort for confirmed enrollment/auth faults (see the Authentication runbook). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSDIAG-Remediation.ps1 -Restart Apply: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Remediation.ps1 -Restart -Confirm:$false (or -Apply) Elevated session required. Use a change window for -Repair / -ReEnroll. EXPECTED RESULTS In preview, a description of the intended action. When applied, the action runs and the script advises re-running NSDIAG-Validation.ps1 after a policy cycle. .NOTES Network category — client troubleshooting toolkit. Restart/repair/re-enroll touch live steering. Netskope's own repair/re-enroll mechanism is vendor-specific — confirm the exact command per Netskope's documentation for your client version (flagged at each action). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$Restart, [switch]$Repair, [switch]$ReEnroll, [switch]$Apply # convenience: equivalent to -Confirm:$false for unattended runs)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSDIAG-Remediation.ps1 -Restart Apply: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Remediation.ps1 -Restart -Confirm:$false (or -Apply) Elevated session required. Use a change window for -Repair / -ReEnroll. EXPECTED RESULTS In preview, a description of the intended action. When applied, the action runs and the script advises re-running NSDIAG-Validation.ps1 after a policy cycle. .NOTES Network category — client troubleshooting toolkit. Restart/repair/re-enroll touch live steering. Netskope's own repair/re-enroll mechanism is vendor-specific — confirm the exact command per Netskope's documentation for your client version (flagged at each action). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$Restart, [switch]$Repair, [switch]$ReEnroll, [switch]$Apply # convenience: equivalent to -Confirm:$false for unattended runs)

NSDIAG-Report.ps1

Purpose: Read-only. Builds the device-context half of the Diagnostic Bundle - Endpoint - Collect: managed-device context from Microsoft Graph (OS, compliance, last sync) so triage can tell a client fault from an enrollment/compliance fault. Also prints the local client summary and a reminder to attach the vendor-deferred client logs for escalation. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSDIAG-Report.ps1 [-DeviceName] [-CsvPath C:\Temp\nsdiag.csv] Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Local client summary, the device's Graph context (compliance/last sync/OS), optional CSV, and a vendor-log handoff reminder. .NOTES Network category — client troubleshooting toolkit. Read-only against Graph. #> [CmdletBinding()] param([string]$DeviceName = $env:COMPUTERNAME, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Report.ps1 [-DeviceName <name>] [-CsvPath C:\Temp\nsdiag.csv] Requires the Microsoft Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Local client summary, the device's Graph context (compliance/last sync/OS), optional CSV, and a vendor-log handoff reminder. .NOTES Network category — client troubleshooting toolkit. Read-only against Graph. #> [CmdletBinding()] param([string]$DeviceName = $env:COMPUTERNAME, [string]$CsvPath)

NSDIAG-Set-ScopeTargeting.ps1

Purpose: Creates or confirms the Entra security group used to scope the toolkit's rollout — 'Diagnostics Scope - Help Desk - Pilot' by default — and reports its membership. SUPPORTS -WhatIf AND DEFAULTS TO PREVIEW: with no confirmation it only describes what it would create. This is the assign-once scope behind the repeatable, run-everywhere model. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSDIAG-Set-ScopeTargeting.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Set-ScopeTargeting.ps1 -Confirm:$false Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS Reports whether the group exists; in preview shows the intended create; when applied, creates the group if missing and prints the group id and current member count. .NOTES Network category — client troubleshooting toolkit. Uses Microsoft Graph (no Netskope internals). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$GroupName = 'Diagnostics Scope - Help Desk - Pilot', [string]$MailNickname = 'diagnostics-scope-helpdesk-pilot')

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSDIAG-Set-ScopeTargeting.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Set-ScopeTargeting.ps1 -Confirm:$false Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS Reports whether the group exists; in preview shows the intended create; when applied, creates the group if missing and prints the group id and current member count. .NOTES Network category — client troubleshooting toolkit. Uses Microsoft Graph (no Netskope internals). #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$GroupName = 'Diagnostics Scope - Help Desk - Pilot', [string]$MailNickname = 'diagnostics-scope-helpdesk-pilot')

NSDIAG-Validation.ps1

Purpose: Read-only. Validates the endpoint's network fundamentals and the client's running state: DNS resolution, TCP reachability to one or more edge hostnames, client service/process up, and a decryption-cert presence check. Produces a clear PASS/FAIL summary for triage. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Validation.ps1 ` -EdgeHost "" -EdgePort 443 The edge hostname/port for your tenant are documented by Netskope — supply them explicitly.

EXPECTED RESULTS DNS, TCP reachability, and client-up checks each report PASS/FAIL; an overall result prints. .NOTES Network category — client troubleshooting toolkit. Read-only. Uses only standard OS facilities. #> [CmdletBinding()] param([string[]]$EdgeHost = @(), [int]$EdgePort = 443, [string]$DnsTestName = 'www.microsoft.com')

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSDIAG-Validation.ps1 ` -EdgeHost "<your-netskope-edge-hostname>" -EdgePort 443 The edge hostname/port for your tenant are documented by Netskope — supply them explicitly. EXPECTED RESULTS DNS, TCP reachability, and client-up checks each report PASS/FAIL; an overall result prints. .NOTES Network category — client troubleshooting toolkit. Read-only. Uses only standard OS facilities. #> [CmdletBinding()] param([string[]]$EdgeHost = @(), [int]$EdgePort = 443, [string]$DnsTestName = 'www.microsoft.com')